Analiza Algoritmilor **Nume & Grupă** _____
Semestrul I
Test 1
05/11/2019
Limită de timp: 90 minute

Question:	1	2	3	4	5	6	7	8	9	10	11	12	Total
Points:	3	3	3	1	1	1	1	1	1	1	1	1	18
Score:													

1. (3 points) Fie funcţia f de mai jos. Determinaţi dacă mulţimea acceptată de această funcţie este R, RE sau NRE.

$$f(P1, P2) = \begin{cases} 1, & \text{dacă P1(w)} = \perp \iff P2(w) \neq \perp, \forall w \\ 0, & \text{altfel} \end{cases}$$

Demonstrăm că mulțimea acceptată de f este NRE, printr-o reducere Turing $\neg PO \leq_T f$. Transformăm (P, w) intrare pentru complementul problemei opririi într-o intrare (P_1, P_2) pentru f a.î. $\neg PO(P, w) = 1 \iff f(P_1, P_2) = 1$

$$P1(x) : \{P(w); return1; \}$$

$$P2(x) : \{return1; \}$$

$\neg PO(P, w) = 1 \implies P(w)$ nu se oprește $\implies P_1(x)$ nu se oprește, $\forall x$. Dar $P_2(x) \neq \perp, \forall x \implies f(P_1, P_2) = 1$

$f(P_1, P_2) = 1$ și $P_2(x) \neq \perp, \forall x \implies P_1(x)$ nu se oprește, $\forall x \implies P(w)$ nu se oprește $\implies \neg PO(P, w) = 1$

Rezultă că $\neg PO \leq_T f$. Știm că mulțimea de adevăr a lui $\neg PO$ este NRE (vezi Observația 1). Astfel mulțimea acceptată de f este NRE.

Observație 1: PO este semidecidabilă, deci mulțimea ei de adevăr este RE si nu este R. Presupunem ca și mulțimea de adevăr a lui $\neg PO$ este RE. Dar această mulțime este complementul mulțimii de adevăr a lui PO, iar din faptul ca ambele sunt RE rezultă că mulțimea de adevăr a lui PO ar fi recursivă, deci PO ar fi decidabilă, contradicție.

Observație: Dacă se demonstrează doar ca mulțimea acceptată de f nu este recursivă, se va primi jumătate din punctaj (1.5 puncte)

2. (3 points) Rezolvați, folosind metoda substituției (inducție), următoarea recurență, după încadrarea ei în clasa de complexitate Θ.

$$T(n) = T(3n/7) + T(4n/7) + n$$

Se dau $T(1) = 1$ și $T(2) = 1$.

Aplicând metoda arborelui de recurență. rezultă un arbore dezechilibrat, care pe ramura din stânga ajunge la T(1) pe nivelul $h_1 = log_{\frac{7}{3}} n$, iar pe ramura din dreapta ajunge la T(1) pe nivelul $h_2 = log_{\frac{7}{4}} n$ (rădăcina arborelui fiind pe nivelul 0). Suma nodurile de pe fiecare din primele h_1 niveluri este n, iar pe următoarele este mai mică decât n, deci avem relația:

$$(h_1 + 1) * n \leq T(n) \leq (h_2 + 1) * n$$

Cum membrul stâng al inegalității este $\Theta(n * log_{\frac{7}{3}} n)$, membrul drept este $\Theta(n * log_{\frac{7}{4}} n)$, iar baza logaritmului nu contează în clasa de complexitate, rezultă $T(n) \in \Theta(n * logn)$. Demonstrăm orin inducție acest rezultat. Trebuie sa găsim c_1, c_2 și n_0 constante a.î. $\forall n \geq n_0$, este adevărată propoziția:

$$P(n) : c_1 * n * logn \leq T(n) \leq c_2 * n * logn$$

În cazul de bază, îl găsim pe n_0. Pentru $n_0 = 1$, avem $P(1) : 0 \leq 1 \leq 0$, fals. Pentru $n_0 = 2$, avem

$$P(2) : 2 * c_1 \leq 1 \leq 2 * c_2 \iff c_1 \leq 1/2 \leq c_2 \ (1)$$

Această relație încheie cazul de bază. Momentan, nu putem spune cât sunt c_1 și c_2, deoarece acestea trebuie să rămână constante pe tot parcursul demonstrației.

Pentru pasul de inducție, presupunem adevărate:

$$P(3n/7) : \ c_1 * \frac{3n}{7} * log(\frac{3n}{7}) \leq T(\frac{3n}{7}) \leq c_2 * \frac{3n}{7} * log(\frac{3n}{7})$$

$$P(4n/7) : \ c_1 * \frac{4n}{7} * log(\frac{4n}{7}) \leq T(\frac{4n}{7}) \leq c_2 * \frac{4n}{7} * log(\frac{4n}{7})$$

Adunând cele două relații, împreună cu un termen n, obținem:

$$c_1 * \frac{3n}{7} * log(\frac{3n}{7}) + c_1 * \frac{4n}{7} * log(\frac{4n}{7}) + n \leq T(n) \leq c_2 * \frac{3n}{7} * log(\frac{3n}{7}) + c_2 * \frac{4n}{7} * log(\frac{4n}{7}) + n$$

Dorim să găsim c_1 și c_2 a.î. să avem simultan:

$$c_1 * n * logn \leq c_1 * \frac{3n}{7} * log(\frac{3n}{7}) + c_1 * \frac{4n}{7} * log(\frac{4n}{7}) + n \ (2)$$

$$c_2 * \frac{3n}{7} * log(\frac{3n}{7}) + c_2 * \frac{4n}{7} * log(\frac{4n}{7}) + n \leq c_2 * n * logn \ (3)$$

Pentru relația (2), avem:

$$(2) \iff c_1 * n * logn \leq c_1 * \frac{3n}{7} * (logn + log(\frac{3}{7})) + c_1 * \frac{4n}{7} * (logn + log(\frac{4}{7})) + n$$

$$\iff c_1 * n * logn \leq c_1 * n * logn + c_1 * n * (\frac{3}{7} * log(\frac{3}{7}) + \frac{4}{7} * log(\frac{4}{7})) + n$$

$$\iff 0 \leq c_1 * n * (\frac{3}{7} * log(\frac{3}{7}) + \frac{4}{7} * log(\frac{4}{7})) + n$$

$$\iff c_1 * (\frac{3}{7} * log(\frac{7}{3}) + \frac{4}{7} * log(\frac{7}{4})) \leq 1$$

$$\iff c_1 \leq \frac{1}{\frac{3}{7} * log(\frac{7}{3}) + \frac{4}{7} * log(\frac{7}{4})} \ (4)$$

Pentru relația (3), avem:

$$(3) \iff c_2 * n * logn \geq c_2 * \frac{3n}{7} * (logn + log(\frac{3}{7})) + c_2 * \frac{4n}{7} * (logn + log(\frac{4}{7})) + n$$

$$\iff c_2 * n * logn \geq c_2 * n * logn + c_2 * n * (\frac{3}{7} * log(\frac{3}{7}) + \frac{4}{7} * log(\frac{4}{7})) + n$$

$$\iff 0 \geq c_2 * n * (\frac{3}{7} * log(\frac{3}{7}) + \frac{4}{7} * log(\frac{4}{7})) + n$$

$$\iff c_2 * (\frac{3}{7} * log(\frac{7}{3}) + \frac{4}{7} * log(\frac{7}{4})) \geq 1$$

$$\iff c_2 \geq \frac{1}{\frac{3}{7} * log(\frac{7}{3}) + \frac{4}{7} * log(\frac{7}{4})} \ (5)$$

Trebuie să arătăm ca există constante care respectă simultan relațiile (1), (4) și (5). Observăm că pentru c_1 am găsit două margini superioare, iar pentru c_2 am găsit două margini inferioare, deci putem alege orice constante care respectă:

$$c_1 \leq min(1/2, \frac{1}{\frac{3}{7} * log(\frac{7}{3}) + \frac{4}{7} * log(\frac{7}{4})}), c_2 \geq max(1/2, \frac{1}{\frac{3}{7} * log(\frac{7}{3}) + \frac{4}{7} * log(\frac{7}{4})})$$

Observații:
1) Nu se va acorda niciun punctaj pentru demonstrarea complexității prin altă metodă
2) Dacă se intuiește, fără demonstrație, complexitatea corectă și se demonstrează corect prin metoda substituției, se va acorda punctajul maxim (3 puncte)
3) Dacă se intuiește o altă clasă de complexitate (care nu poate fi tot Θ) și se demonstează corect prin metoda substituției, se va acorda 1 punct (de exemplu, pentru $T(n) \in O(n^2)$ sau $T(n) \in \Omega(n)$
4) Dacă se demonstrează corect doar una din inegalitățile din rezolvare (adică $T(n) \in O(nlogn)$ sau $T(n) \in \Omega(nlogn)$, se vor acorda 2 puncte.

3. (3 points) Rezolvați prin orice metodă.

$$T(n) = 4T(\sqrt{n}) + log^2(n)$$

Facem schimbarea de variabilă $n = 2^m$ și obținem

$$T(2^m) = 4 * T(2^{\frac{m}{2}}) + log^2(2^m)$$

Notăm $S(m) = T(2^m)$ și avem

$$S(m) = 4 * S(m/2) + m^2$$

Încercăm să aplicăm teorema Master. Avem a = 4, b = 2, $f(m) = m^2 \implies m^{log_b(a)} = m^2 \implies f(m) \in \Theta(m^{log_b(a)})$. Suntem în cazul 2 al teoremei Master, cu k = 0, deci $S(m) \in \Theta(m^2 * logm) \implies T(2^m) \in \Theta(m^2 * logm)$. Ținând cont că $n = 2^m$ (deci $m = logn$), rezultă $T(n) \in \Theta(log^2 n * log(logn))$.

Observație: Orice rezolvare care conduce la rezultatul corect va fi punctată (de exemplu, metoda iterației)

1 Grile

În secţiunea următoare sunt prezentare întrebări de tip grilă cu un singur răspuns corect. Găsiţi rezultatul şi notaţi-l în tabelul de la final.
Observație: În barem, am ordonat variantele de răspuns astfel încât răspunsul corect este mereu primul
Răspunsurile corecte pentru varianta 1: 4 - d), 5 - a), 6 - d), 7 - b), 8 - a), 9 - c), 10 - d), 11 - c), 12 - b)
Răspunsurile corecte pentru varianta 2: 4 - c), 5 - c), 6 - d), 7 - c), 8 - b), 9 - b), 10 - c), 11 - d), 12 - c)

4. (1 point) Fie mulţimile A, B şi C, despre care ştim: A este recursivă, $A \cup B$ nu este recursivă şi $B \cup C$ este recursiv-enumerabilă. Ce se poate spune despre $A \cup C$?

a) $A \cup C$ poate fi oricum
b) $A \cup C$ este R
c) $A \cup C$ este RE, dar nu R
d) $A \cup C$ este non-RE

Nu putem spune nimic despre $A \cup C$. Dăm exemple de mulțimi A, B și C pentru toate variantele (R, RE, NRE).
$A = \varnothing, B \in RE \setminus R, C = \varnothing \implies A \cup C \in R$
$A = \varnothing, B \in RE \setminus R, C = \in RE \setminus R \implies A \cup C \in RE \setminus R$
$A = \varnothing, B \in RE \setminus R, C = N \setminus B \implies A \cup C \in NRE$

5. (1 point) Fie recurenţa $T(n) = 2T(n/4) + \sqrt{n} * log(n) + \sqrt{n}$. Ce se poate spune despre apartenenţa ei la o clasă de complexitate?

   a) $T(n) \in \Theta(\sqrt{n} * log^2(n))$
   b) $T(n) \in \Theta(n * log^2(n))$
   c) $T(n) \in \Theta(\sqrt{n} * log(n))$
   d) $T(n) \in \Theta(n\sqrt{n})$

   Aplicăm teorema Master, cazul 2, cu k = 1.

6. (1 point) Fie recurenţa $T(n) = T(n/3)+T(n/6)+\Theta(n^{\sqrt{log(n)}})$. Ce se poate spune despre apartenenţa ei la o clasă de complexitate?

   a) $T(n) \in \Theta(n^{\sqrt{log(n)}})$
   b) $T(n) \in \Theta(n^{\sqrt{n}})$
   c) $T(n) \in \Theta(n * log(n))$
   d) $T(n) \in \Theta(n^{\sqrt{log(n)}} log(n))$

   Deoarece T(n) este o funcție de complexitate, avem $T(n/3) \geq T(n/6)$, deci

   $$2T(n/6) + f(n) \leq T(n) \leq 2T(n/3) + f(n),$$

   unde $f(n) = \Theta(n^{\sqrt{log(n)}})$. Atât membrul stâng, cât și membrul drept al inegalității de mai sus se încadrează în cazul 3 al teoremei Master, deci au complexitatea $\Theta(f(n)) \implies T(n) \in \Theta(n^{\sqrt{log(n)}})$.

7. (1 point) Fie recurenţa $T(n) = 2\sqrt{n} * T(\sqrt{n}/3) + n * log(n)$. în ce caz al teoremei Master se încadrează?

   a) Niciunul
   b) Cazul I
   c) Cazul al II-lea
   d) Cazul al III-lea

   Pentru a putea aplica teorema Master, recurența trebuie să fie de forma $T(n) = aT(n/b) + f(n)$, cu a și b constante. În cazul nostru, $a = 2\sqrt{n}$, care nu este o constantă.

8. (1 point) Fie funcţiile $f(n) = 100n + \sqrt{n}$ şi $g(n) = n + log^2(n)$. Care este cea mai restrictivă încadrare pe care o putem face?

   a) $f(n) \in \Theta(g(n))$
   b) $f(n) \in \omega(g(n))$
   c) $f(n) \in O(g(n))$
   d) $f(n) \in \Omega(g(n))$

   Ambele funcții sunt $\Theta(n)$ (în clasa de compexitate se păstrează doar termenul dominant)

9. (1 point) Fie funcţiile $f(n) \in \Omega(n^2)$ şi $g(n) = o(n*log(n))$. Care este cea mai restrictivă încadrare pe care o putem face pentru $f(n)/g(n)$?

   a) $f(n)/g(n) \in \omega(n/log(n))$
   b) $f(n)/g(n) \in \Omega(n/log(n))$
   c) $f(n)/g(n) \in o(n/log(n))$
   d) $f(n)/g(n) \in \Omega(n^2)$

   Aplicăm definițiile claselor de complexitate:

   $$\exists c_1 > 0, n_0 \in N \text{ a.î. } f(n) \geq c_1 * n^2, \forall n \geq n_0$$

   $$\forall c > 0, \exists n(c) \in N \text{ a.î. } g(n) < c * n * log(n), \forall n \geq n(c)$$

   Împărțind aceste relații, obținem:

   $$\forall c' > 0, \forall n \geq n'(c'), f(n)/g(n) > c' * (n/log(n)),$$

   unde $c' = c_1/c, n'(c') = max(n_0, n(c))$. Această relație este chiar definiția clasei de complexitate $\omega(n/log(n))$.

10. (1 point) Care este complexitatea următoarei bucăți de cod?

```
for (int i=n; i>0; i/ =3) do
    for (int j=1; j<n; j* =2) do
        for (int k=i; k<n; k++) do
            ... //număr constant c1 de operații
        end for
    end for
end for
```

    a) $\Theta(n * log^2(n))$
    b) $\Theta(n^3)$
    c) $\Theta(n * log^3(n))$
    d) $\Theta(n^2 * log(n))$

    For-ul după j este independent de celelalte, și contribuie la complexitatea finală cu $\Theta(logn)$. Trebuie să numărăm de câte ori se intră în total în for-ul după k. Pentru fiecare valoare a lui i, vom intra în for-ul după k de n - i ori, deci complexitatea celorlalte două for-uri este:

    $$\sum_{l=0}^{log_3(n)} (n - 3^l) = n * (log_3(n) + 1) - \frac{3^{log_3(n)+1} - 1}{2} = \Theta(nlogn)$$

    Înmulțind cu logn menționat mai devreme (for-urile sunt imbricate), rezultă complxitatea finală $\Theta(n * log^2(n))$

11. (1 point) Care este complexitatea următoarei bucăți de cod?

```
for (int i=1; i<n; i*=2) do
    for (int j=i; j>=1; j/=2) do
        for (int k=0; k<i; k++) do
            ... //număr constant c1 de operații
        end for
    end for
end for
```

a) $\Theta(n * log(n))$
b) $\Theta(n * log^2(n))$
c) $\Theta(log^3(n))$
d) $\Theta(n^2)$

Toate cele 3 for-uri sunt dependente între ele. Aflăm complexitatea finală calculând de câte ori se intră în for-ul după k. Pntru fiecare valoare a lui i, se intră în for-ul după j de log(i) + 1 ori. Pentru fiecare valoare a lui j, se intră în for-ul după k de i ori. Înmulțim aceste valori, deoarece for-urile sunt imbricate. Deoarece i ia valori puteri de 2, notăm $l = log_2(i)$ și obținem complexitatea totală:

$$S = \sum_{l=0}^{log_2(n)} l * 2^l = 1 * 2^1 + 2 * 2^2 + ... + log_2(n) * 2^{log_2(n)}$$

Notăm $S_a = \sum_{l=a}^{log_2(n)} 2^l = 2^a + ... + 2^{log_2(n)}$ și observăm că $S = S_1 + S_2 + ... + S_{log_2(n)}$. Calculăm separat fiecare sumă:

$$S_a = 2^{log_2(n)+1} - 2^a = 2 * n - 2^a$$

Avem $log_2(n)$ astfel de relații, pe care le însumăm și obținem complexitatea totală

$S = 2 * n * log_2(n) - (2^1 + 2^2 + ... + 2^{log_2(n)}) = 2 * n * log_2(n) - 2 * n + 2 \in \Theta(n * log(n))$

12. (1 point) Care dintre următoarele probleme nu este semi-decidabilă?

a) Problema $\neg PO$, exprimată prin funcţia

$$g(P, w) = \begin{cases} 1, & \text{pentru P(w) } \perp \\ 0, & \text{pentru P(w) } \neq \perp \end{cases}$$

b) Problema PO, exprimată prin funcţia

$$f(P, w) = \begin{cases} 1, & \text{pentru P(w) } \neq \perp \\ 0, & \text{pentru P(w) } \perp \end{cases}$$

c) PCP
d) Niciuna dintre celelalte opţiuni

Din Observația 1 de la problema 1 $\implies \neg PO$ nu este semi-decidabilă. PO și PCP sunt semi-decidabile (vezi seminarul 2).